

Práctica: Monitoreo de un clúster AWS EMR con Prometheus y Grafana

Miguel Sedano Izurieta

IABD

Big Data Aplicado

1. Configuración clúster EMR	3
1.1 Creación del clúster	3
1.2 Exposición del puerto 7070	4
1.3 Conexión y configuración del nodo maestro	4
2. Configuración de JMX Exporter	5
2.1 ¿Qué es JMX?	5
2.2 ¿Qué papel tiene JMX en un clúster AWS EMR?	5
2.3 ¿Por qué es útil en este entorno?	5
2.4 Conclusión	6
2.5 Crear el archivo de configuración.....	6
2.6 Configurar el NameNode para usar JMX Exporter	6
2.7 Reiniciar el NameNode	7
3. Despliegue de Prometheus y Grafana.....	8
3.1 Crear una instancia EC2 para Prometheus y Grafana	8
3.2 Instalar Prometheus	8
3.3 Configurar Prometheus.....	8
3.4 Instalar Grafana	9
4. Visualización de métricas en Grafana	11
5. Preguntas para reflexión	13
1. ¿Qué métricas consideras más importantes para monitorear en un clúster EMR? ¿Por qué?	13
2. ¿Cómo podrías mejorar la configuración de JMX Exporter para recopilar métricas más específicas?	13
3. ¿Qué ventajas tiene usar Prometheus y Grafana frente a otras herramientas de monitoreo?.....	14

1. Configuración clúster EMR

1.1 Creación del clúster

Resumen Información

Nombre y aplicaciones - obligatorio

Nombre
EMR-Monitoring-iabd11

Versión de Amazon EMR
emr-6.6.0

Paquete de aplicaciones
Custom (Hadoop 3.2.1, Hive 3.1.2, Spark 3.2.0)

Configuración del clúster - obligatorio

Grupos de instancias uniformes
Principal (m4.large), Central (m4.large)

Aprovisionamiento y escalado de clústeres - obligatorio

Configuración de aprovisionamiento
Tamaño del núcleo: 2 instancias

Redes - obligatorio

VPC
vpc-0f9fe37fe... [\[?\]](#)

Subred
subnet-01e1bc... [\[?\]](#)

Grupos de seguridad de nodos principales
sg-0c325a0b1e... [\[?\]](#)

Grupos de seguridad de nodos básicos
sg-0682434216... [\[?\]](#)

Terminación del clúster

Terminación del clúster
Terminar el clúster después del tiempo de inactividad
Tiempo de inactividad: 4 horas

Registros de clúster

Ubicación de Amazon S3
s3://aws-logs... [\[?\]](#)

Terminación del clúster
Terminar el clúster después del tiempo de inactividad
Tiempo de inactividad: 4 horas

Registros de clúster

Ubicación de Amazon S3
s3://aws-logs... [\[?\]](#)

Configuración de seguridad y par de claves de EC2

Par de claves de Amazon EC2
vockey [\[?\]](#)

Roles de Identity and Access Management (IAM) - obligatorio

Rol de servicio
LabRole [\[?\]](#)

Perfil de instancia
EMR_EC2_DefaultRole [\[?\]](#)

2. Configuración de JMX Exporter

2.1 ¿Qué es JMX?

JMX (Java Management Extensions) es una tecnología propia del ecosistema Java que permite gestionar y monitorizar aplicaciones Java en tiempo de ejecución. JMX proporciona una forma estandarizada de acceder a métricas, estadísticas y configuraciones internas de una aplicación Java sin tener que modificar su código.

En términos prácticos, JMX actúa como una interfaz de monitoreo y gestión de los componentes internos de una aplicación Java, utilizando objetos llamados MBeans (Managed Beans).

2.2 ¿Qué papel tiene JMX en un clúster AWS EMR?

Un clúster AWS EMR suele estar compuesto por herramientas del ecosistema Hadoop, que están escritas en Java. Esto significa que todos estos servicios exponen internamente métricas a través de JMX.

Por ejemplo, el NameNode de HDFS, el ResourceManager de YARN o los procesos de Spark permiten consultar información como:

- Uso de CPU y memoria.
- Número de bloques en HDFS.
- Estado de los nodos del clúster.
- Métricas de rendimiento de las tareas MapReduce o Spark.

Estas métricas están disponibles a través de endpoints JMX, pero en un formato que no es directamente legible por herramientas modernas como Prometheus.

2.3 ¿Por qué es útil en este entorno?

En el contexto del monitoreo de un clúster EMR, el JMX Exporter es fundamental por varias razones:

1. **Accesibilidad:** Permite que Prometheus recopile métricas detalladas sin necesidad de modificar el código de los servicios Hadoop o Spark.
2. **Visibilidad en tiempo real:** Las métricas se actualizan en tiempo real y pueden ser visualizadas mediante dashboards en Grafana.
3. **Alertas proactivas:** Prometheus puede usar estas métricas para generar alertas cuando se detectan comportamientos anómalos (como bajo rendimiento del NameNode, uso elevado de RAM, etc.).

4. Escalabilidad y mantenimiento: Ayuda a los equipos de administración a tomar decisiones basadas en datos para escalar, optimizar o diagnosticar el clúster.

2.4 Conclusión

JMX en este entorno sirve como la capa de introspección de los servicios Java del clúster EMR. El JMX Exporter permite que esta información se recoja, almacene y visualice con Prometheus y Grafana. Sin esta integración, sería mucho más difícil monitorear el estado y el rendimiento del clúster de manera eficaz.

```
[hadoop@ip-172-31-53-195 ~]$ wget https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
--2025-04-11 16:05:54-- https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
Resolving repo1.maven.org (repo1.maven.org)... 146.75.32.209, 2a04:4e42:78::209
Connecting to repo1.maven.org (repo1.maven.org)[146.75.32.209]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 469645 (459K) [application/java-archive]
Saving to: 'jmx_prometheus_javaagent-0.16.1.jar'

100%[=====] 469,645 --.-K/s in 0.003s

2025-04-11 16:05:54 (146 MB/s) - 'jmx_prometheus_javaagent-0.16.1.jar' saved [469645/469645]
```

2.5 Crear el archivo de configuración

Creamos el archivo config.yml en el cual agregamos el siguiente contenido:

```
GNU nano 2.9.8 config.yml Modified
lowercaseOutputName: true
lowercaseOutputLabelNames: true
rules:
- pattern: ".*"
```

2.6 Configurar el NameNode para usar JMX Exporter

Editamos el archivo de configuración del NameNode en el que incluimos la línea de comando que se observa en el pantallazo

```
hadoop@ip-172-31-53-195:~$ nano /etc/hadoop/conf/hadoop-env.sh
GNU nano 2.9.8 /etc/hadoop/conf/hadoop-env.sh Modified

export HADOOP_CLASSPATH="$HADOOP_CLASSPATH:/usr/share/aws/emr/goodies/lib/emr-hadoop-goodies.jar"

export HADOOP_CLASSPATH="$HADOOP_CLASSPATH:/usr/share/aws/emr/kinesis/lib/emr-kinesis-hadoop.jar"

# Add CloudWatch sink jar to classpath
export HADOOP_CLASSPATH="$HADOOP_CLASSPATH:/usr/share/aws/emr/cloudwatch-sink/lib/*"

# Add security artifacts to classpath
export HADOOP_CLASSPATH="$HADOOP_CLASSPATH:/usr/share/aws/emr/security/conf:/usr/share/aws/emr/security/lib/*"

export HADOOP_OPTS="$HADOOP_OPTS -server -XX:+ExitOnOutOfMemoryError"
export HADOOP_NAMENODE_HEAPSIZE=1024
export HADOOP_DATANODE_HEAPSIZE=614
export HADOOP_JOB_HISTORYSERVER_HEAPSIZE=2252

#Configuración necesaria para la tarea
export HADOOP_NAMENODE_OPTS="-javaagent:/home/hadoop/jmx_prometheus_javaagent0.16.1.jar=12345:/home/hadoop/config.yml $$"

^G Get Help      ^O Write Out     ^W Where Is      ^K Cut Text      ^J Justify       ^C Cur Pos       M-U Undo         M-A Mark Text
^X Exit          ^R Read File     ^\ Replace       ^U Uncut Text    ^T To Linter     ^_ Go To Line     M-E Redo         M-6 Copy Text
```

2.7 Reiniciar el NameNode

Utilizamos el siguiente comando para reiniciar el servicio y que se apliquen los cambios.

```
[hadoop@ip-172-31-53-195 ~]$ sudo systemctl restart hadoop-hdfs-namenode
[hadoop@ip-172-31-53-195 ~]$
```

3. Despliegue de Prometheus y Grafana

3.1 Crear una instancia EC2 para Prometheus y Grafana

Resumen de instancia de i-Odea8f564041831d8 (EC2Monitoring-iabd11) [Información](#)

[Conectar](#) [Estado de la instancia](#) [Acciones](#)

Se ha actualizado hace less than a minute

ID de la instancia i-Odea8f564041831d8	Dirección IPv4 pública 54.237.237.215 dirección abierta	Direcciones IPv4 privadas 172.31.27.123
Dirección IPv6 -	Estado de la instancia En ejecución	DNS de IPv4 pública ec2-54-237-237-215.compute-1.amazonaws.com dirección abierta
Tipo de nombre de anfitrión Nombre de IP: ip-172-31-27-123.ec2.internal	Nombre DNS de IP privada (solo IPv4) ip-172-31-27-123.ec2.internal	Direcciones IP elásticas -
Responder al nombre DNS de recurso privado IPv4 (A)	ID de VPC vpc-0f9fe37fe2d5659f9	Hallazgo de AWS Compute Optimizer Suscribirse a AWS Compute Optimizer para recibir recomendaciones. Más información
Dirección IP asignada automáticamente 54.237.237.215 [IP pública]	ID de subred subnet-0fa6ff2475477cbb3	Nombre del grupo de Auto Scaling -
Rol de IAM -	ARN de instancia arn:aws:ec2:us-east-1:873371097384:instance/i-Odea8f564041831d8	Administradas falso
IMDSv2 Required		

3.2 Instalar Prometheus

Utilizamos el siguiente comando para instalar Prometheus

```
ubuntu@ip-172-31-27-123:~$ wget https://github.com/prometheus/prometheus/releases/download/v2.30.3/prometheus-2.30.3.linux-amd64.tar.gz
--2025-04-11 16:35:40-- https://github.com/prometheus/prometheus/releases/download/v2.30.3/prometheus-2.30.3.linux-amd64.tar.gz
Resolving github.com (github.com)... 140.82.114.4
Connecting to github.com (github.com)|140.82.114.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/bc9e0970-09b3-4893-a66b-5fb918691a1e?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250411%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250411T163541Z&X-Amz-Expires=300&X-Amz-Signature=60f17d58e7443e29afc4e64af36cb81c10321b958f03aa1ccc00073ecab1400&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.30.3.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream [following]
--2025-04-11 16:35:41-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/bc9e0970-09b3-4893-a66b-5fb918691a1e?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250411%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250411T163541Z&X-Amz-Expires=300&X-Amz-Signature=60f17d58e7443e29afc4e64af36cb81c10321b958f03aa1ccc00073ecab1400&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.30.3.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.108.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 72638078 (69M) [application/octet-stream]
Saving to: 'prometheus-2.30.3.linux-amd64.tar.gz'

prometheus-2.30.3.linux-amd64 100%[=====] 69.27M 107MB/s in 0.6s

2025-04-11 16:35:41 (107 MB/s) - 'prometheus-2.30.3.linux-amd64.tar.gz' saved [72638078/72638078]

ubuntu@ip-172-31-27-123:~$
```

3.3 Configurar Prometheus

Utilizamos los siguientes comandos para acceder al archivo prometheus.yml.

```
ubuntu@ip-172-31-27-123:~$ tar -xzf prometheus-2.30.3.linux-amd64.tar.gz
ubuntu@ip-172-31-27-123:~$
```



```
ubuntu@ip-172-31-27-123:~$ cd prometheus-2.30.3.linux-amd64
ubuntu@ip-172-31-27-123:~/prometheus-2.30.3.linux-amd64$ ./prometheus --config.file=prometheus.yml
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:400 msg="No time or size retention was set so using the default time retention" duration=15d
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:438 msg="Starting Prometheus" version="(version=2.30.3, branch=HEAD, revision=f29cacc42557f6a8ec30ea9b3c8c089391bd5df)"
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:443 build_context="(go=go1.17.1, user=root@5cfff4265f0e3, date=2021-10-05-16:10:52)"
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:444 host_details="(Linux 6.8.0-1024-aws #26-Ubuntu SMP Tue Feb 18 17:22:37 UTC 2025 x86_64 ip-172-31-27-123 (none))"
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:445 fd_limits="(soft=1024, hard=1048576)"
level=info ts=2025-04-11T16:37:00.722Z caller=main.go:446 vm_limits="(soft=unlimited, hard=unlimited)"
level=info ts=2025-04-11T16:37:00.724Z caller=web.go:541 component=web msg="Start listening for connections" address=0.0.0.0:9090
```

Dentro de la línea añadimos el cluster de EMR como objetivo

```
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
  - job_name: "emr-namenode"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

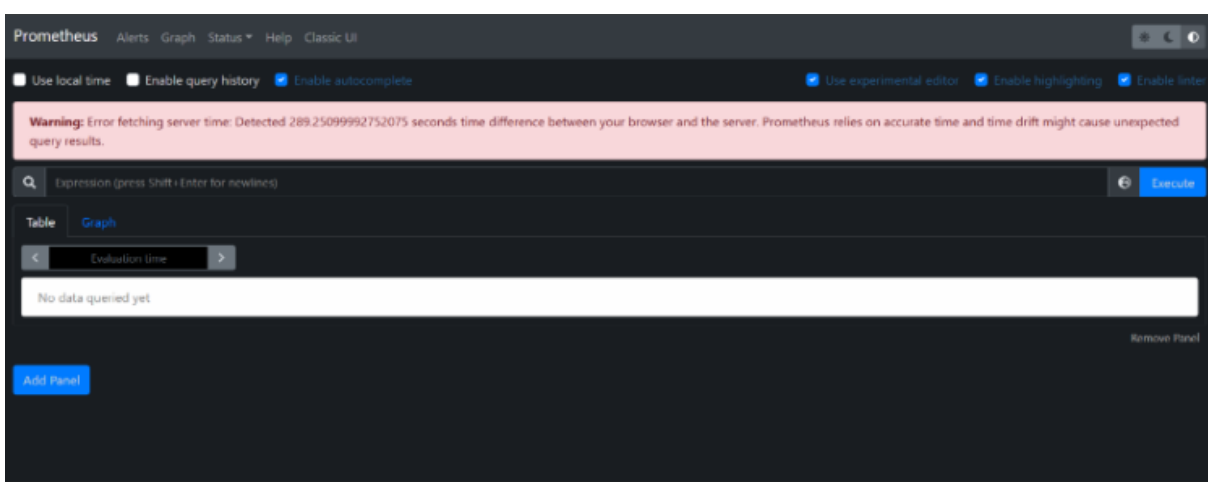
static_configs:
  - targets: ["54.123.45.67:12345"]
```

3.4 Instalar Grafana

Usamos los siguientes códigos para instalar Grafana

```
ubuntu@ip-172-31-27-123:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install -y apt-transport-https
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'apt' instead of 'apt-transport-https'
apt is already the newest version (2.7.14build2).
apt set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-27-123:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install -y software-properties-common wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.49.1).
software-properties-common set to manually installed.
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-27-123:~/prometheus-2.30.3.linux-amd64$
```

A continuación, accedemos a la interfaz de Prometheus:



Instalamos Grafana con los siguientes comandos:

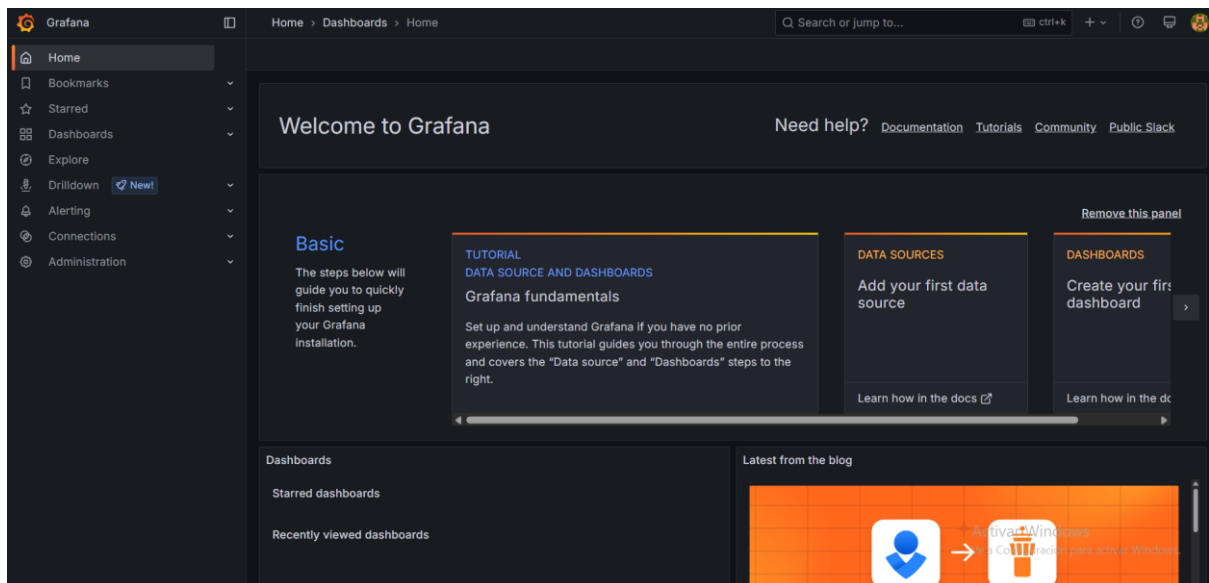
```

ubuntu@ip-172-31-27-123:~$ sudo apt-get install grafana
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package grafana
ubuntu@ip-172-31-27-123:~$ sudo apt-get install -y apt-transport-https
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apt-transport-https is already the newest version (2.7.14build2).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
ubuntu@ip-172-31-27-123:~$ sudo apt-get install -y software-properties-common wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.49.2).
wget is already the newest version (1.21.4-ubuntu4.1).
0 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
ubuntu@ip-172-31-27-123:~$ wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
ubuntu@ip-172-31-27-123:~$ echo "deb https://packages.grafana.com/oss/deb stable main"
deb https://packages.grafana.com/oss/deb stable main
ubuntu@ip-172-31-27-123:~$ sudo tee -a /etc/apt/sources.list.d/grafana.list

ubuntu@ip-172-31-27-123:~$ echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee -a /etc/apt/sources.li
st.d/grafana.list
deb https://packages.grafana.com/oss/deb stable main
ubuntu@ip-172-31-27-123:~$ sudo apt-get update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 https://packages.grafana.com/oss/deb stable InRelease [7661 B]
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:6 https://packages.grafana.com/oss/deb stable/main amd64 Packages [379 kB]
Fetched 387 kB in 1s (615 kB/s)
Reading package lists... Done
W: https://packages.grafana.com/oss/deb/dists/stable/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/tr
usted.gpg), see the DEPRECATION section in apt-key(8) for details.
ubuntu@ip-172-31-27-123:~$ sudo apt-get install grafana
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  musl
The following NEW packages will be installed:
  grafana musl
0 upgraded, 2 newly installed, 0 to remove and 57 not upgraded.
Need to get 169 MB of archives.
After this operation, 631 MB of additional disk space will be used.

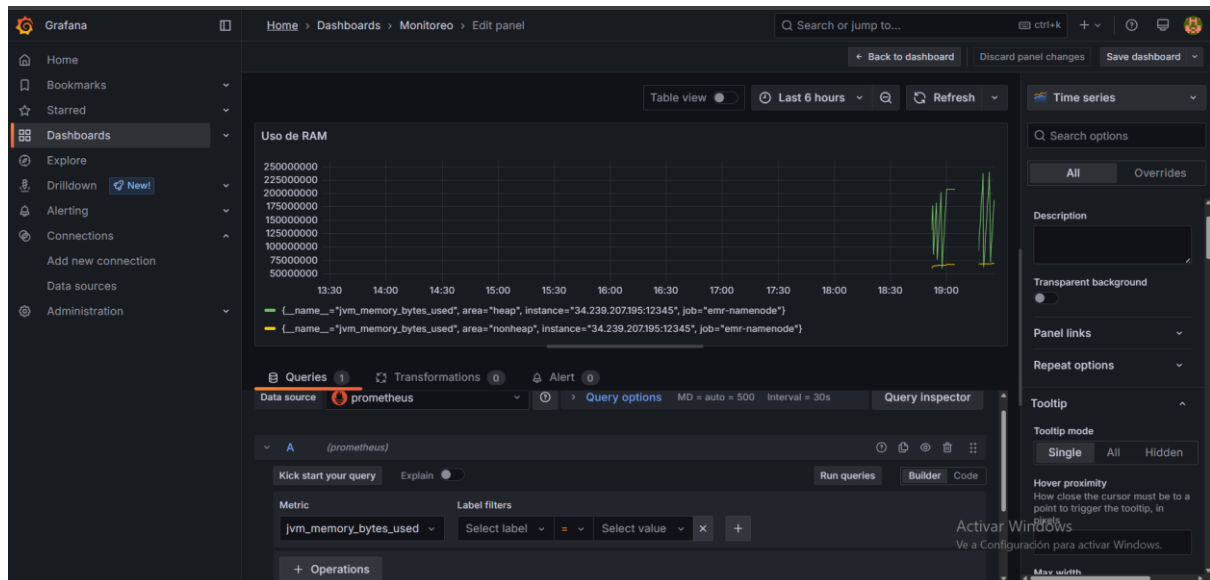
```

A continuación, la interfaz de Grafana:



4. Visualización de métricas en Grafana

En nuestro perfil debemos añadir un nuevo Dashboard, y dentro de este una nueva visualización. Una vez hecho simplemente añadiremos el nombre de las métricas que correspondan en cada visualización. Esta es la pantalla donde se configuran las visualizaciones:

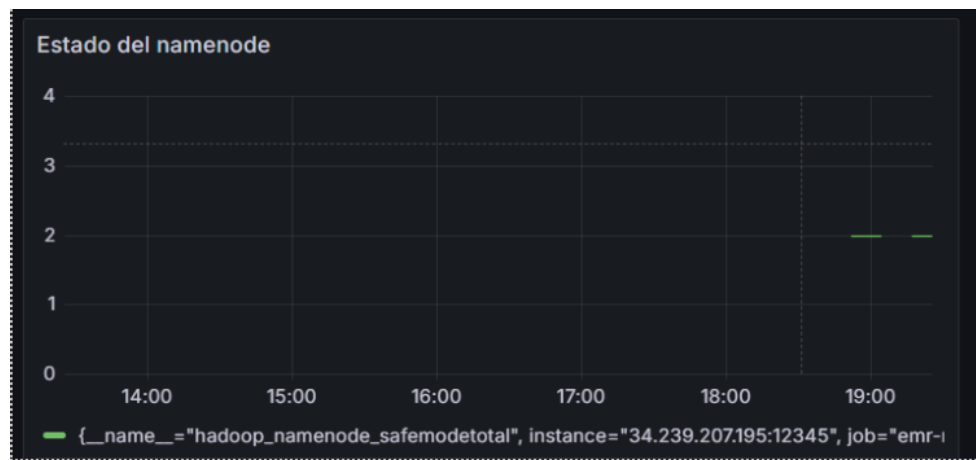


Así es como se ven las visualizaciones de las métricas requeridas en la tarea:

- Uso de RAM



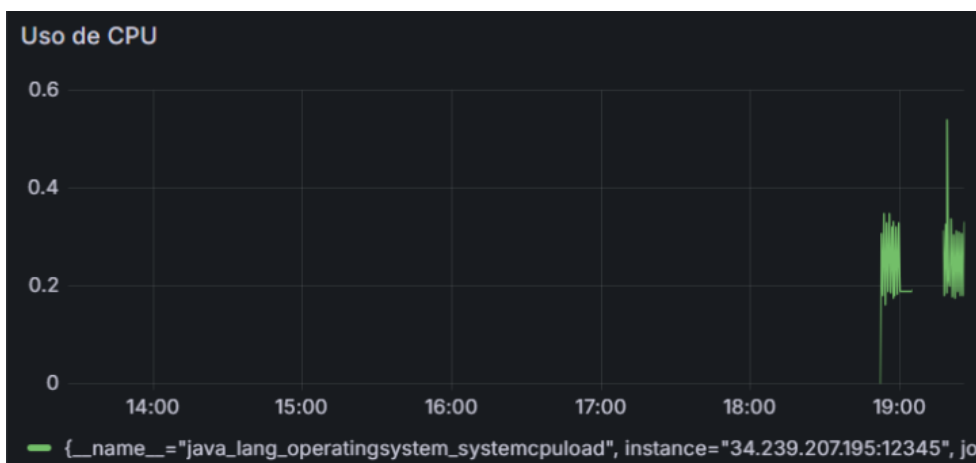
- Estado del NameNode:



- Espacio usado en HDFS:



- Uso de CPU:



5. Preguntas para reflexión

1. ¿Qué métricas consideras más importantes para monitorear en un clúster EMR? ¿Por qué?

En un clúster EMR, algunas de las métricas más importantes a monitorear son:

- Uso de CPU y memoria (RAM) de cada nodo: Para detectar cuellos de botella y asegurar que los recursos se están utilizando eficientemente. Un alto uso sostenido puede indicar que es necesario escalar el clúster.
- Espacio utilizado en HDFS: Es crucial saber cuánto almacenamiento está disponible y cuánto se está utilizando para evitar fallos por falta de espacio.
- Estado del NameNode y DataNodes: El NameNode es un punto crítico en HDFS, por lo que conocer su disponibilidad y estado es vital. Si el NameNode cae, el clúster no podrá acceder al sistema de archivos.
- Tiempos de ejecución de trabajos Spark o Hive: Estas métricas ayudan a analizar el rendimiento de los trabajos y a detectar tareas que tarden demasiado o fallen con frecuencia.

Estas métricas permiten tener una visión clara del estado de salud, rendimiento y eficiencia del clúster, ayudando tanto en la detección temprana de problemas como en la toma de decisiones para ajustes o escalado.

2. ¿Cómo podrías mejorar la configuración de JMX Exporter para recopilar métricas más específicas?

La configuración básica del JMX Exporter suele incluir una regla genérica (`- pattern: ".*"`), que captura todas las métricas disponibles, pero no permite control ni organización sobre lo que se recolecta. Para mejorar esta configuración, se pueden aplicar las siguientes acciones:

- Definir reglas específicas en el archivo `config.yml`, usando expresiones regulares para capturar solo las métricas relevantes, evitando ruido innecesario.
- Agregar etiquetas (labels) personalizadas a las métricas para distinguir entre diferentes nodos o servicios.
- Filtrar métricas duplicadas o irrelevantes que puedan saturar Prometheus o dificultar la visualización en Grafana.
- Separar la configuración por componente, usando distintos puertos o archivos de configuración para el NameNode, ResourceManager, etc., lo que facilita la gestión modular del monitoreo.

Con estas mejoras, se obtiene un conjunto de métricas más limpio, más útil y mejor estructurado para el análisis en Prometheus y la visualización en Grafana.

3. ¿Qué ventajas tiene usar Prometheus y Grafana frente a otras herramientas de monitoreo?

Prometheus y Grafana se han convertido en una de las combinaciones más populares en monitoreo moderno por varias razones:

- Código abierto y gratuito: A diferencia de herramientas comerciales como Datadog o New Relic, Prometheus y Grafana no tienen costos asociados por licencias o número de nodos.
- Integración sencilla con sistemas basados en Java (como Hadoop o Spark), mediante el uso del JMX Exporter.
- Alta personalización: Grafana permite construir dashboards personalizados de manera muy visual y dinámica, adaptados a las necesidades específicas del sistema.
- Escalabilidad y rendimiento: Ambas herramientas están diseñadas para funcionar en entornos distribuidos y a gran escala, lo que las hace ideales para clústeres como los de AWS EMR.

En resumen, Prometheus y Grafana ofrecen potencia, flexibilidad y una curva de aprendizaje razonable, lo que los convierte en una excelente elección para monitorear infraestructuras modernas.

Espacio usado en HDFS



Uso de CPU

